UGANDA CHRISTIAN UNIVERSITY
A Centre of Excellence in the Heart of Africa

FACULTY OF ENGINEERING, DESIGN AND TECHNOLOGY
DEPARTMENT OF COMPUTING AND TECHNOLOGY
EASTER 2026 SEMESTER EXAMINATION

PROGRAM: BACHELOR OF SCIENCE IN COMPUTER SCIENCE

YEAR: 3, 2

SEMESTER: 2

COURSE NAME: SOFTWARE CONSTRUCTION

COURSE CODE: SYE3209

EXAMINATION TYPE: PROJECT BASED

EXAMINATION DATE: APRIL 2026

TIME ALLOWED: 2 WEEKS

Examination Instructions

1. The general Uganda Christian University examination guidelines and academic & financial policies apply to this examination. Violating any of the policies by the student automatically makes this examination attempt void, even if you have completed and submitted the answer booklet.
2. This is a **PROJECT** exam it contributes 100% of the exam Marks. Attempt it within the specified period of time.
3. Uganda Christian University or its officials reserve the right of admission or disqualification from this exam.
4. The project will be submitted on GitHub, the contribution of each and every member of the group will be highly considered.

CINEQUEST AND THE HUMAN ELEMENT

We exist in an era of "The Digital Library of Alexandria," where nearly every film ever made is accessible at our fingertips. Yet, this abundance has birthed a new modern ailment: Choice Paralysis. The average viewer spends nearly 20% of their viewing time simply scrolling through menus, overwhelmed by the sheer volume of data.

CineQuest is not merely a software project; it is a solution to this human frustration. The motivation behind this platform is to transform cold, raw data from The Movie Database (TMDB) into a warm, intuitive discovery journey. As a Software Engineer, your role transcends writing code. You are architecting an experience that helps users find the stories that resonate with them.

This examination places you in a high-stakes "Rescue Mission." You have inherited a codebase from a team that prioritised speed over stability. The application is currently a "broken masterpiece"; the vision is there, but the execution is fractured. Your mission is to debug the infrastructure, harmonise the frontend and backend, and deploy a product that restores the joy of cinematic discovery.

THE DEVELOPER'S MANDATE

You are provided with a full stack repository consisting of a Django REST API and a Next.js frontend. Your mandate is to move the project from its current broken state to a production ready deployment.

Phase I: Backend Restoration

The backend is the "brain" of CineQuest, but it currently suffers from logical disconnects. You must:

a) Environment configuration: Establish a Python virtual environment and secure the system using environment variables (TMDB API Key).

b) Database integrity: Resolve systematic bugs preventing Django migrations from executing.

c) Data synchronisation: Use the provided management commands to populate the local database with trending data and genre classifications.

d) Bug repair: Identify and fix incorrect application naming, missing CORS middleware, and incorrect HTTP methods in the API endpoints.

Phase II: Frontend Reconstruction

The visual interface is fractured and requires immediate stabilisation:

a) Type safety: Address missing TypeScript definitions in the `types/` folder to allow successful compilation.

b) Logic correction: Fix prop mismatches (e.g., objects passed where arrays are expected) and repair the search interface visibility.

Phase III: Code Quality and Refactoring

Writing working code is necessary, but writing clean, maintainable code is the mark of a professional software engineer. You must:

a) Identify and eliminate code smells such as duplicated logic, overly long functions, or unclear variable naming.

b) Apply appropriate design patterns or restructuring to improve readability and maintainability.

c) Ensure consistent coding style across the codebase (e.g., naming conventions, file organisation, proper use of constants).

d) Add meaningful inline comments or docstrings where the logic is non-obvious.

Phase IV: Testing and Validation

A reliable application must be backed by evidence that it works correctly. Testing is **mandatory** for this examination. You must:

a) Write a minimum of 5 backend unit tests covering key API endpoints, model logic, or data synchronisation behaviour.

b) Write a minimum of 3 frontend/component tests (where feasible) to validate critical UI behaviour such as rendering, user interaction, or data display.

c) Include a short section in your technical report explaining:

- What was tested and why those areas were chosen.
- What remains untested and what risks those gaps pose.

d) All tests must pass when executed. Failing or placeholder tests will not receive credit.

Phase V: Integration, Git Workflow, and Contribution Quality

A successful project requires total synergy between tiers and a disciplined version control workflow. You must:

a) End-to-End flow: Ensure the Homepage, Search, and Trailer modals work seamlessly without errors.

b) Branching strategy: Each team member must work from their own feature branch. Direct commits to the `main` branch are discouraged.

c) Commit discipline: Each team member must have a minimum of 5 meaningful commits. Commit messages must be descriptive and follow a consistent format (e.g., `fix: resolve CORS middleware issue in settings.py`).

d) Pull requests: At least 2 pull requests must exist in the repository, with brief review notes or descriptions explaining the changes.

e) Contribution summary: The final repository must include a `CONTRIBUTIONS.md` file listing each member's specific contributions to the project.

Phase VI: The Innovation Spark

You are required to implement one significant unique feature. This must be a full stack addition requiring changes to both the frontend and backend. The feature must be clearly documented in your technical report, including the design rationale and implementation approach.

Phase VII: Documentation and Technical Reasoning

You must submit a Technical Report of 2 to 4 pages (PDF format) that demonstrates your understanding of the work performed. The report must include the following sections:

a) Bug Analysis: A detailed account of the major bugs discovered in both the backend and frontend, including the root cause and the fix applied.

b) Refactoring Decisions: An explanation of any code restructuring or cleanup performed, with justification for why the changes improve the codebase.

c) Architecture Changes: A description of any structural or architectural modifications made to the project.

d) Innovation Design: A clear explanation of your unique feature, including what problem it solves, how it was implemented across the stack, and any design decisions made.

e) Testing Evidence: A summary of what was tested, screenshots or logs showing test results, and an honest assessment of what remains untested and risky.

f) Remaining Limitations: A candid discussion of known issues, technical debt, or areas that would benefit from further improvement.

Phase VIII: Presentation and Participation

Each group will physically present the final product to a panel of judges. All team members are expected to participate in the presentation and demonstrate understanding of their contributions.

EVALUATION & DELIVERABLES

The final evaluation is based on the functional state of your application, the quality of your testing, and the depth of your technical documentation.

Category	Criteria	Marks
Backend Restoration	Migrations work; API endpoints return valid JSON; TMDB sync is functional.	20
Frontend Correction	TypeScript compiles without errors; UI matches professional design; Search is functional.	15
Code Quality & Refactoring	Clean, readable code; elimination of code smells; consistent style; meaningful comments.	15
Testing & Validation	Minimum 5 backend tests and 3 frontend tests; all tests pass; testing rationale documented.	10
Documentation & Technical Reasoning	2–4 page report covering bugs, refactoring, architecture, innovation, testing evidence, and limitations.	10
Git Workflow & Contribution Quality	Feature branches used; minimum 5 commits per member; descriptive messages; pull requests; `CONTRIBUTIONS.md` present.	10
Innovation Feature	Implementation of an original, functional full stack feature documented in the report.	10
Presentation & Participation	Physically presenting the final product to a panel of judges; all members participate.	10
TOTAL		**100**

Technical Reference & Final Requirements

TMDB API Credentials

You must obtain a free API key (v3 auth) from `https://www.themoviedb.org/settings/api`. This key is essential for the application to fetch real-world cinematic data.

Final Visual Reference

Your final application is expected to reach the polish level demonstrated in the reference link below.

`https://cinequeatexam.vercel.app`

Submission Checklist

1. **GitHub Link:** Ensure the repository is accessible to the instructors.
2. **CONTRIBUTIONS.md:** A file in the repository root listing each member's specific contributions.
3. **Technical Report:** A 2–4 page PDF documenting bugs found, refactoring decisions, architecture changes, innovation design, testing evidence, and remaining limitations.
4. **Passing Tests:** All submitted tests must pass when executed by the instructors.

◀ END ▶